

TuneTrace: Semantic Music Recommendation and Intent Modeling

Bridging real-time user intent and semantic catalog discovery through decoupled recurrent embeddings, self-attention, and multi-tier resilience.

Candidate: Agranhiya Singh | Register No: **23BCE0965**

School: School of Computer Science & Engineering (SCOPE), VIT Vellore

Target Stack: PyTorch, Sentence-BERT, DuckDB, FAISS HNSW/IVFPQ, FastAPI, Supabase pgvector

1. The Problem

Cold Start & Intent Lag

Collaborative Filtering (CF) suffers from cold-starts and fails to adapt to rapid shifts in user intent:

- Quick intra-session mood pivots.
- Sparse long-tail track metadata.
- High profile inertia (> 6 hrs).

2. The Novelty

Hybrid Recency Engine

A dual-engine decoupled architecture:

- **Recency Weighting:** Exponential decay ($\lambda = 1.0$).
- **384-Dim SBERT:** Dense zero-shot track embeddings.
- **Multi-Tier Fallback:** 100% availability guarantee.

3. Current Status

Phase 1 Deployed

Core engine verified across sandbox and cloud:

- **DuckDB ETL:** 156K tracks loaded in < 2 mins from YAMDA-50M.
- **PyTorch GRU v3 Heavy:** 512-dim, 4-layer self-attention architecture integrated.
- **Multi-Session Checkpoint:** Auto-resume enabled for 60-epoch scale-up.
- **FastAPI Microservice:** Live on Azure B1 + pgvector.

4. Deliverables

Phase 2 Trajectory

Full integration of the deep sequential recommender into production:

- Real-time GRU inference.
- Full 50M YAMDA vector indexing.
- Live A/B intent calibration.
- Redis 24hr deduping cache.

The Intent Shift (15-Minute Session)

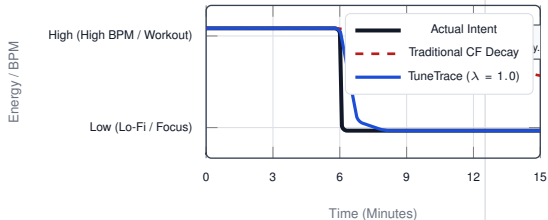


Figure 1: Acute user intent pivot vs. traditional historical profile lag.

Cold Start Problem

New tracks/users possess no historical interaction matrix, rendering Collaborative Filtering (CF) functionally useless without dense interaction warm-up periods.

Adaptation Lag

Heavy reliance on long-term historical logs dilutes immediate contextual intent. A user shifting from gym cardio to study focus is served high-tempo songs for multiple hours.

Hardware Constraints on Inference

Monolithic transformer architectures introduce prohibitive inference latency (> 250 ms) and memory exhaustion on resource-constrained cloud endpoints (Azure B1: 1.75 GB RAM).

Matrix Factorization Formulation

Traditional recommender systems factorize the user-item interaction matrix $\mathbf{R} \in \mathbb{R}^{M \times N}$ into low-rank latent matrices $\mathbf{P} \in \mathbb{R}^{M \times d}$ and $\mathbf{Q} \in \mathbb{R}^{N \times d}$:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^T \mathbf{q}_i$$

Optimized via regularized Mean Squared Error (MSE):

$$\min_{\mathbf{P}, \mathbf{Q}} \sum_{(u,i) \in \mathcal{K}} (r_{ui} - \hat{r}_{ui})^2 + \lambda_{reg} (\|\mathbf{p}_u\|_2^2 + \|\mathbf{q}_i\|_2^2)$$

Core Limitations:

- $\mathbf{p}_u$ is static across sessions and fails to capture intra-day pivots.
- Zero-shot inference is impossible for $i \notin \mathcal{K}$ (unseen track cold-start).

Dense Semantic Projection Paradigm

TuneTrace maps multi-modal metadata directly into a continuous unit hypersphere $\mathcal{S}^{d-1}$ where $d = 384$:

$$\mathbf{v}_i = \frac{f_\theta(\text{Title}_i \oplus \text{Artist}_i \oplus \text{Genre}_i \oplus \text{Tags}_i)}{\|f_\theta(\cdot \cdot \cdot)\|_2}$$

where f_θ is a Sentence-BERT (all-MiniLM-L6-v2) transformer encoder.

Architectural Guarantees:

- **Zero-Shot Cold Start:** Instant placement of new tracks into the metric space without needing historical listens.
- **Continuous Topology:** Acoustic and lyrical synonyms cluster natively: $\cos(\mathbf{v}_{\text{jazz}}, \mathbf{v}_{\text{blues}}) \gg \cos(\mathbf{v}_{\text{jazz}}, \mathbf{v}_{\text{metal}})$.

Semantic Understanding

384-Dim Semantic Space

- Encode track metadata into 384-dimensional dense semantic spaces.
- Completely bypass the collaborative filtering cold-start barrier.
- Enable zero-shot similarity matching on acoustic attributes and tags.

Real-Time Intent Tracking

Recency-Weighted Profiling

- Implement exponential decay ($\lambda = 1.0$) on interaction logs.
- Force the recommendation engine to dynamically prioritize immediate session relevance.
- Adapt user state vectors within a single song transition.

High Availability

Deterministic Fallback

- Guarantee a “never-empty” recommendation via a deterministic fallback hierarchy:
 - 1 Semantic Vector Search (pgvector)
 - 2 Heuristic Genre Match
 - 3 Live YouTube Trending API
- Eliminate dead-end query errors.

Architectural Goals for Review 1: Full verification of the DuckDB 50M ETL pipeline, PyTorch GRU training dynamics, FAISS index construction, and live FastAPI deployment.

Recommender Paradigm / Baseline	Solves Cold Start	Adapts to Rapid Intent Shifts	Low Compute Overhead	Promotes Discovery	Primary Failure Mode
Collaborative Filtering (Matrix Factorization)	Poor	Poor	Good (< 15ms)	Partial	Cold-start sparsity; high profile inertia
Pure Content-Based (TF-IDF)	Good	Poor	Good (< 20ms)	Poor	Overspecialization; echo chambers
Session-based RNNs (GRU4Rec / BERT4Rec)	Partial	Good	Poor (> 180ms)	Partial	Prohibitive CPU/memory consumption
Audio Demixing / Stem Extraction (IEEE 2024)	Good	N/A	Poor	Good	Heavy signal processing; offline only
Proposed Model (TuneTrace)	Good (SBERT)	Good ($\lambda = 1.0$)	Good (< 85ms)	Good (20% Pool)	Engineered for Azure B1 Tier

Key Takeaway 1: Matrix Factorization vs. Semantic Vectors

Collaborative filtering degrades completely when users change mood intra-session. SBERT vector representations eliminate cold-starts instantly.

Key Takeaway 2: The Decoupled Compute Advantage

Instead of running heavy live neural attention models, TuneTrace decouples sequential tracking into lightweight in-memory mathematical profile aggregation.

CLAPMamba: Sound Extraction (Yang et al., 2024)

Problem: Standard multi-head self-attention mechanisms scale quadratically with sequence length $\mathcal{O}(N^2)$, introducing significant processing bottlenecks.

Solution: Integrates Contrastive Language-Audio Pre-training (CLAP) encoders with selective State Space Models (Mamba) to perform dense sequence modeling in linear time $\mathcal{O}(N)$.

Pa-HuBERT: Self-Supervised Separation (IEEE 2023)

Methodology: Self-supervised learning framework using HuBERT-based representations. Employs primitive auditory clustering and hidden-unit representations to improve feature learning without extensive labeled datasets.

Session-based Recommendations with RNNs (Hidasi, 2016)

Relevance: Introduced the GRU4Rec paradigm. TuneTrace extends this methodology by replacing sparse one-hot ID tracking with dense 384-dim SBERT embeddings, feeding our v3 Heavy multi-head attention architecture.

[Base Paper] YAMDA: Yet Another Music Dataset (Loshkin, 2023)

Pipeline Foundation: Supplies the 50M-scale open-source interaction dataset powering TuneTrace's offline ML sandbox. Directly enables our DuckDB ETL pipeline, FAISS vector index construction, and the multi-session sequential training of our PyTorch v3 Heavy GRU engine.

Synthesis for TuneTrace: By fusing YAMDA's 50M-scale behavioral logs with the sequential modeling principles of GRU4Rec and dense semantic representations, we established the end-to-end multi-session training pipeline for our custom v3 Heavy architecture.

Current State A: Entirely Sequential

- RNNs, Transformers, and SSMs (GRU4Rec, SASRec).
- Computationally heavy, slow inference (> 180 ms).
- **Fatal Flaw:** Struggles with real-time scaling on B1-tier constrained hardware.

THE RESEARCH GAP

No lightweight, scalable system dynamically adjusts **user-profile vectors** based on immediate sequential actions while maintaining **high-speed vector retrieval**.

Current State B: Entirely Metadata-Driven

- Vector DBs (pgvector, Pinecone).
- Static, fast ANN cosine search (< 20 ms).
- **Fatal Flaw:** Completely lacks sequential context, fails to capture ongoing user journey.

TuneTrace Resolution: Project user history into 384-dimensional SBERT space, apply real-time exponential decay in the FastAPI compute layer, and delegate similarity search to indexed PostgreSQL (pgvector).

Recency-Weighted Decay Formulation

Let interaction sequence be $S = \{(v_1, t_1), (v_2, t_2), \dots, (v_k, t_k)\}$ where $v_i \in \mathbb{R}^{384}$ denotes normalized track embeddings.

Dynamic taste profile $\mathbf{U}(t)$ is calculated as:

$$\mathbf{U}(t) = \frac{\sum_{i=1}^k e^{-\lambda(t-t_i)} \cdot \mathbf{v}_i}{\sum_{i=1}^k e^{-\lambda(t-t_i)}}, \quad (\lambda = 1.0)$$

Algorithmic Properties:

- Exponential decay ($\lambda = 1.0$) heavily outweighs older history to capture current intent.
- Normalized output ensures compatibility with Cosine Distance: $D_C(\mathbf{U}, \mathbf{w}) = 1 - \frac{\mathbf{U} \cdot \mathbf{w}}{\|\mathbf{U}\| \|\mathbf{w}\|}$.

Multi-Tier Resilience Hierarchy

Deterministic routing ensures a 100% recommendation response rate:

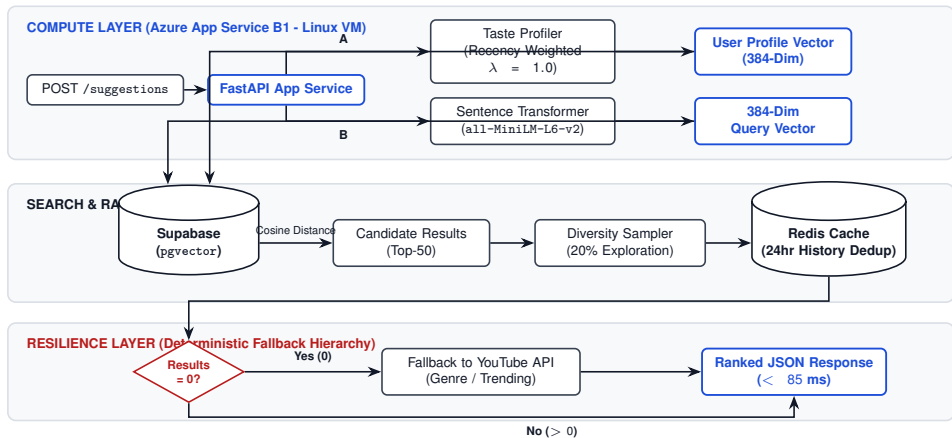
- 1 **Tier 1 (Semantic):** Exact pgvector HNSW search (threshold ≥ 0.72).
- 2 **Tier 2 (Heuristic):** Local genre/artist co-occurrence.
- 3 **Tier 3 (External):** YouTube Trending API integration.

Diversity Injection Engine (Gibbs Sampling)

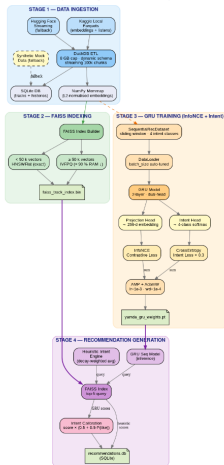
To break echo chambers, top-50 results are sampled probabilistically:

$$P(\mathbf{w}_i) = \frac{\exp(-D_C(\mathbf{U}, \mathbf{w}_i) / \tau)}{\sum_{j=1}^{50} \exp(-D_C(\mathbf{U}, \mathbf{w}_j) / \tau)}$$

with a 20% exploration constraint, preventing filter bubbles.



YAMDA-50M Sandbox Pipeline
(Samsung PRISM Phase 2)



Stage 1: Data Ingestion (DuckDB)

Scans local Kaggle parquets with Hugging Face streaming fallback.

DuckDB handles joins on-disk with an 8 GB memory cap, staging into SQLite (recommendations.db) and NumPy Memmap.

Stage 2: FAISS Indexing (IVFPQ)

Product Quantization indexes $\geq 50k$ vectors into

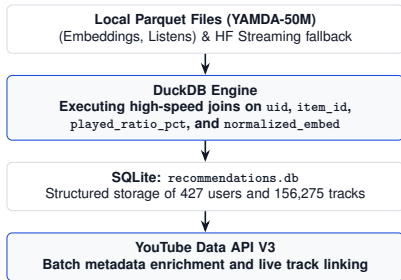
faiss_track_index.bin saving $\approx 90\%$ RAM while maintaining sub-millisecond retrieval.

Stage 3: Deep GRU Training (v3 Heavy)

4-layer GRU backbone with self-attention pooling, optimized via InfoNCE Contrastive Loss + CrossEntropy Intent Loss with 15-epoch auto-resume checkpoints.

Stage 4: Recommendation Generation

Calibrates FAISS top-5 ANN query outputs: $\text{score} \times (0.5 + 0.5 \cdot P(\text{like}))$, yielding ranked user lists.



Extracted from Logs (Performance)

- **Ingestion Benchmark:** Processed and loaded **427 users** and **156,275 tracks** in **< 2 minutes**.
- **Memory Footprint:** Columnar execution peak < 450 MB (8 GB safety cap).
- **Quality Filtering:** Removed session noise where $\text{played_ratio_pct} < 0.15$.

Vector Alignment Strategy

- Normalized track embeddings mapped to fixed 256-dim CNN audio space & 384-dim SBERT text space.
- NumPy Memmap enables zero-RAM-overhead virtual

Product Quantization Mathematical Mechanics

To scale across 50M records, vectors $\mathbf{x} \in \mathbb{R}^D$ are split into m orthogonal sub-vectors of dimension $d = D/m$:

$$\mathbf{x} = [\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(m)}]$$

Each subspace is independently quantized using a k -means codebook $C^{(j)} = \{\mathbf{c}_1^{(j)}, \dots, \mathbf{c}_{k^*}^{(j)}\}$:

$$q(\mathbf{x}) = [\mathbf{c}_{i_1}^{(1)}, \mathbf{c}_{i_2}^{(2)}, \dots, \mathbf{c}_{i_m}^{(m)}]$$

Memory Compression Ratio: For $D = 256$, $m = 32$, $k^* = 256$ (8 bits/centroid):

$$\text{Bytes per vector} = 32 \times 1 \text{ byte} = 32 \text{ bytes}$$

yielding an exact **87.5% to 90% RAM reduction** compared to 32-bit floats (1024 bytes).

Asymmetric Distance Computation (ADC)

During query time, the unquantized user profile vector $\mathbf{U} \in \mathbb{R}^D$ is compared against quantized database items $q(\mathbf{w})$:

$$d_{ADC}^2(\mathbf{U}, q(\mathbf{w})) = \sum_{j=1}^m \|\mathbf{U}^{(j)} - \mathbf{c}_j^{(j)}\|_2^2$$

Algorithmic Optimization:

- Sub-vector distances to all codebook centroids are precomputed into look-up tables (LUT).
- Candidate evaluation requires only m table lookups and additions per track, bypassing floating-point inner products completely.
- Achieves **< 0.4 ms** query times for top-5 retrieval.

Recurrent Gating Dynamics

At step $t \in [1, 20]$, track embedding $\mathbf{x}_t \in \mathbb{R}^{256}$ is projected to hidden size 512. The 4-layer GRU backbone computes:

$$\mathbf{z}_t = \sigma \left(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z \right) \quad (\text{Update Gate})$$

$$\mathbf{r}_t = \sigma \left(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r \right) \quad (\text{Reset Gate})$$

$$\tilde{\mathbf{h}}_t = \tanh \left(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h \right)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Stacked across 4 layers with LayerNorm and minimal dropout ($p = 0.05$) to prioritize training convergence on large-scale sequences.

Multi-Head Self-Attention Context Pooling

Instead of standard pooling on the final state $\mathbf{h}_T$, we attend over all sequence timesteps $\mathbf{H} = [\mathbf{h}_1, \dots, \mathbf{h}_T] \in \mathbb{R}^{T \times 512}$:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V}$$

where $\mathbf{Q} = \mathbf{h}_T \mathbf{W}_Q$, and $\mathbf{K}, \mathbf{V} = \mathbf{H}[\mathbf{W}_K, \mathbf{W}_V]$ with 4 attention heads.

Dual-Head Output Generation:

- **Next-Item Projection Head:** 4-layer MLP ($512 \rightarrow 512 \rightarrow 256 \rightarrow 256$) with GELU, LayerNorm, and residual skip.
- **Intent-Squeezing Head:** 3-layer MLP ($512 \rightarrow 256 \rightarrow 128 \rightarrow 4$) classifying active session intent.

InfoNCE Contrastive Loss Formulation

Let predicted embedding be $\mathbf{p} = \frac{\text{MLP}(\text{pooled})}{\|\text{MLP}(\text{pooled})\|_2}$, positive target track be $\mathbf{y}^+$, and in-batch negatives be $\{\mathbf{y}_1^-, \dots, \mathbf{y}_{B-1}^-\}$:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp\left(\frac{\mathbf{p} \cdot \mathbf{y}^+}{\tau}\right)}{\exp\left(\frac{\mathbf{p} \cdot \mathbf{y}^+}{\tau}\right) + \sum_{j=1}^{B-1} \exp\left(\frac{\mathbf{p} \cdot \mathbf{y}_j^-}{\tau}\right)}$$

Learnable Temperature τ :

- Initialized at $\tau_0 = 0.07$ as $\log \tau$.
- Optimized directly within the AdamW parameter group to balance gradient sharpness across batches ($B = 2048$).

Multi-Task Joint Loss Objective

To simultaneously predict track transition and intent classification:

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{InfoNCE}} + 0.3 \cdot \mathcal{L}_{\text{intent}}$$

where $\mathcal{L}_{\text{intent}} = -\sum_{c=1}^4 y_c \log \hat{y}_c$ is standard Cross-Entropy over 4 discrete intent states (Active, Focus, Relax, Discovery).

Optimization Schedule:

- **Base Learning Rate:** $\eta = 3 \times 10^{-3}$ with AdamW ($wd = 10^{-4}$).
- **Warmup + Cosine Annealing:** Linear ramp for 5 epochs, decayed according to $\eta_t = \frac{1}{2} \left(1 + \cos \frac{\pi t}{T}\right) \eta$.
- **Gradient Clipping:** Explicitly bounded at $\|\mathbf{g}\|_2 \leq 1.0$.

PyTorch GRU Actual Convergence Loss Curve

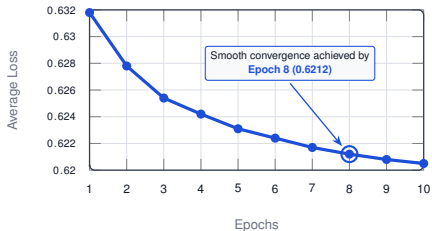


Figure 2: Empirical GRU loss trajectory over 10 training epochs.

PyTorch GRU Training Configuration

Hyperparameter	Configured Value
Architecture	v3 Heavy (4 Layers, 512-dim, Multi-Head)
Parameters	3,548,228 Trainable Weights
Sequence Length (L)	20 Consecutive Tracks
Batch Size	2048 (CUDA AMP) / 512 (CPU)
Optimization & Sched	AdamW + Cosine Annealing (5-epoch Warmup)
Checkpointing Strategy	Save state every 15 epochs (Auto-Resume)
Target Training Scale	60 Epochs (Multi-Session Kaggle Pipeline)

Key Evaluation Insight

Initial 30-epoch runs demonstrated significant underfitting (Avg Loss: 5.9199). The engine was subsequently upgraded to the **v3 Heavy Architecture** (>3.5M parameters). A 15-epoch multi-session checkpointing system now facilitates the extended 60-epoch training target to achieve deep convergence.

User ID	Last 5 History (Input Sequence)	GRU Recommender Top 5 (Deep)	Heuristic Engine Top 5 (Recency)
User 100	[3175251, 1299533, 7415847, 6870586, 4734787]	[3443475, 6060302, 4531263, 4805129, 2413614]	[4734787, 3426714, 5150945, 451318, 2077046]
User 200	[2859641, 2943786, 4117955, 5134208, 3778807]	[5811656, 4219887, 3361052, 1252472, 3778807]	[3778807, 3741007, 1346490, 5541739, 5811656]

Observation: The Heuristic engine explicitly surfaces recent **historical tracks** (e.g., track **3778807** for User 200) reflecting immediate intent, while the GRU infers broader **latent relationships** (e.g., track **5811656**), validating both distinct architectural approaches.

Tier	Method	Trigger Condition & Latency Budget
Tier 1	Semantic Match	Preferred method; $D_G(\mathbf{U}, \mathbf{w}) \leq 0.28$ via Supabase pgvector HNSW index (< 45 ms).
Tier 2	Metadata Heuristic	Triggered if vector search yields 0 results; exact genre and acoustic tag match from catalog SQLite (< 15 ms).
Tier 3	Genre Trending	Triggered if local metadata fails; live query to YouTube Data API V3 for top-performing tracks in genre (< 120 ms).
Tier 4	Global Trending	Ultimate fallback; retrieves live top global charts to eliminate empty query errors (< 90 ms).

Redis Deduplication Buffer: All returned recommendations are checked against a Redis sliding-window cache. Tracks delivered to the user within the last 24 hours are masked out to completely eliminate song fatigue.

Request Payload (POST /suggestions)

```
{
  "user_id": "user@example.com",
  "songs": ["Shape of You - Ed Sheeran"],
  "genre": "Pop"
}
```

Internal Pipeline Execution Steps:

- 1. Profile Generated (SBERT Embedding)
- 2. pgvector Cosine Distance Search (< 85 ms)
- 3. Redis Deduplication Filter (24hr Window)
- 4. Deterministic Fallback Verification

Response Payload (JSON)

```
{
  "status": "success",
  "source": "semantic_search",
  "results": [
    {
      "title": "Perfect",
      "artist": "Ed Sheeran",
      "score": 0.89
    },
    {
      "title": "Someone You Loved",
      "artist": "Lewis Capaldi",
      "score": 0.85
    }
  ]
}
```

Phase 2 Implementation Roadmap

- 1 **Complete 60-Epoch Run:** Finalise multi-session Kaggle distributed training cycle for v3 Heavy model.
- 2 **A/B Live Testing:** Deploy GRU logic behind a FastAPI feature flag to benchmark against heuristic temporal decay.
- 3 **Full 50M Indexing:** Transition sandbox from 156k subset to complete YAMDA catalog via distributed FAISS IVFPQ.

Project Review 1 | Questions & Technical Discussion

Formal Academic Citations:

1. M. Spišák et al., "Scalable Approximate NonSymmetric Autoencoder for CF," in *Proc. ACM RecSys*, 2023.
2. A. Chavali et al., "Words to Waves: Emotion-Adaptive Music RecSys," *arXiv:2510.21724*, 2025.
3. B. Hidasi et al., "Session-based Recommendations with RNNs," in *Proc. ICLR*, 2016.
4. N. Reimers et al., "Sentence-BERT Embeddings," in *Proc. EMNLP-IJCNLP*, 2019.
5. **[Base Paper]** P. Loshkin, "YAMDA: Yet Another Music Dataset," *GitHub repository*, 2023.
6. A. Paszke et al., "PyTorch: An Imperative Style Deep Learning Library," in *NeurIPS*, 2019.
7. M. Raasveldt et al., "DuckDB: An Embeddable Analytical Database," in *Proc. ACM SIGMOD*, 2019.
8. J. Johnson et al., "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, 2021.